

Securing Agentic AI Systems

Agentic systems — multi-step reasoning pipelines, tool-using agents, autonomous workflows — introduce attack surfaces that single-shot LLM calls do not. The core shift: the model no longer just produces text; it **acts**, in a loop, on state it partly controls, often reading data an attacker can plant.

Why agents are different (the novel attack surface)

1. **Compounding trust.** Each step's output becomes the next step's input. One injected instruction early can propagate through the whole chain.
2. **Indirect injection is the main threat.** The agent reads tool outputs, web pages, files, emails. Any of these can carry instructions (ATLAS AML.T0051.001). The attacker is often not the user.
3. **Confused deputy.** The agent runs with the *service's* permissions, which are usually broader than the *requesting user's*. Without per-user scoping, an attacker borrows the agent's authority.
4. **Tool composition.** Individually safe tools can be chained into an exfiltration path (read secret → send via "fetch URL").
5. **Runaway loops.** Reasoning loops can spin without termination — burning budget (denial of wallet, LLM10) or repeatedly hammering a tool.
6. **Memory persistence.** A poisoned fact written to long-term memory re-injects on every future run.

Controls

Scoped permissions (least agency)

Default-deny allow-list of (tool, resource) the agent may touch, scoped to the *end user's* entitlements. The model proposes a tool call; the guard decides.

```
from airte.guardrails import AgentGuard, Permission, ToolCallRequest
guard = AgentGuard(
    permissions=[Permission("search"), Permission("read_doc", frozenset({"kb"}))],
    max_steps=15, max_tool_calls=30,
    require_approval_for=frozenset({"send_email", "delete_record"})
guard.authorize(ToolCallRequest("search"))          # ALLOW
guard.authorize(ToolCallRequest("delete_record"))    # raises HumanApprovalRequired
```

Human-in-the-loop for irreversible actions

Money movement, data deletion, identity/permission changes, and external communication should require explicit human approval. This contains both honest bugs and successful attacks, because the manipulated agent still can't complete the high-impact step alone.

Step and cost budgets (loop + DoS guard)

Hard caps on reasoning steps, total tool calls, per-tool rate, tokens, and spend. `AgentGuard.tick()` raises `StepBudgetExceeded` when a loop runs away.

Treat all retrieved/tool content as untrusted data

Fence and quarantine anything the agent reads before it enters the reasoning context, so instructions hidden in data are neutralized:

```
from airte.guardrails import RAGContext
ctx = RAGContext(allowed_sources=frozenset({"kb", "tickets"}))
fenced = ctx.build([(doc_text, "kb")]) # tagged UNTRUSTED, quarantined if risky
```

Isolation and reversibility

Run tool execution in a sandbox with egress denied by default. Prefer reversible operations and dry-run modes; require approval to cross from dry-run to commit.

Observability and kill switch

Log every step, tool call, and guard decision with a correlation ID. Maintain a kill switch: feature-flag tools off, revoke the agent's identity, rotate keys. Treat a compromised agent as a compromised service account.

Putting it in the loop

```
while not done:
    guard.tick() # loop guard
    proposal = model.next_action(context) # untrusted planner
    guard.authorize(proposal.request, # least privilege + HITL
                    human_approved=ask_if_needed(proposal))
    result = execute(proposal) # sandboxed action plane
    guard.record(proposal.request)
    context += fence_untrusted(result) # data, not instructions
```

The invariant: **authority is enforced by the deterministic guard, never assumed from the model's cooperation.**